MOKENGELI

Plateforme d'intelligence artificielle prédictive
anti-fraude en temps réel

Cahier des charges & document de conception

Hackathon **RawBank** — Finale
Mars 2026

Équipe projet — Brainsoft

DEBUZE David

Louis

Ruth

Simbi

Strategie

Document généré pour la soumission et le développement du prototype MOKENGELI.

Table des matières

1 Signification du nom **1**

1.1 Identité du produit 1

1.2 Ancrage RawBank et contexte 1

1.3 Les trois notions clés 1

2 Vision du projet et principe général **2**

2.1 Mission fonctionnelle 2

2.2 Objectifs mesurables (indicateurs de succès) 2

2.3 Contraintes et principes directeurs 2

3 Les trois modèles : rôle et fonctionnement **4**

3.1 Modèle 1 — Scoring transactionnel (supervisé) 4

3.1.1 Rôle 4

3.1.2 Fonctionnement 4

3.1.3 Type de ML 4

3.1.4 Données d'entraînement 4

3.1.5 Variables importantes (exemples) 5

3.1.6 Sortie 5

3.2 Modèle 2 — Détection d'anomalie de session 5

3.2.1 Rôle 5

3.2.2 Fonctionnement 5

3.2.3 Type de ML 5

3.2.4 Données 5

3.2.5 Variables importantes 5

3.2.6 Sortie 6

3.3 Modèle 3 — Comportement du titulaire (profil habituel) 6

3.3.1 Rôle 6

3.3.2 Fonctionnement 6

3.3.3 Type de ML 6

3.3.4 Sortie 6

3.4 Dataset unifié : 20 features et 3 cibles 6
3.4.1 Principe 6
3.4.2 Les trois cibles 7
3.4.3 Liste des 20 features 7
3.4.4 Répartition indicative par modèle 7
3.4.5 Exemple de cinq enregistrements (synthétique) 8

4 Combinaison des trois modèles 9
4.1 Logique de fusion 9
4.2 Exemples de politique (à calibrer) 9
4.3 Rôle de la couche de règles métier 9

5 Structure de la base de données 10
5.1 Principe d'accès 10
5.2 Tables principales et rôle métier 10
5.3 Relations cardinales 10
5.4 Colonnes clés rappelées 10

6 Architecture de l'application 12
6.1 Backend (Node.js) 12
6.2 Frontend (React) 12
6.3 Base Supabase 12
6.4 Sécurité dans la stack 12
6.5 Organisation du dépôt (monorepo) 12

7 Fonctionnalités complètes (détail) 13
7.1 Fonctionnalité 1 — Scoring transactionnel en millisecondes 13
7.2 Fonctionnalité 2 — Politique et file d'attente 13
7.3 Fonctionnalité 3 — Alertes synchronisées 13
7.4 Fonctionnalité 4 — Boucle d'apprentissage 13
7.5 Fonctionnalité 5 — Tableau de bord temps réel 13
7.6 Fonctionnalité 6 — Score de session à la connexion (RAW Sentinel ID) 14
7.7 Fonctionnalité 7 — Risque relationnel bénéficiaire (RAW Graph Vigil) 14
7.8 Fonctionnalité 8 — Step-up dynamique (RAW Trust Engine) 14

8 Interfaces utilisateur — vue d'ensemble 15

9 Présentation des interfaces (fenêtres des applications) 16

9.1 Application client — simulation bancaire 16

9.2 Application serveur — back-office 16

9.3 Cohérence 16

10 Utilisation du logiciel et scénarios de test 17

10.1 Acteurs 17

10.2 Chaîne d'intégration 17

10.3 Option A — Simulation complète (prioritaire) 17

10.4 Option B — Sandbox RawBank 17

10.5 Option C — Démo hors ligne 17

11 Flux des données et exemple de traitement 18

11.1 Séquence runtime 18

11.2 Exemple chiffré (C-502) 18

12 Sécurité 19

12.1 Transport et secrets 19

12.2 Authentification et autorisation 19

12.3 Données et validation 19

12.4 Résilience 19

12.5 Chaîne ML 19

12.6 PKI, mTLS et intégrité des payloads 19

13 Annexes techniques et gouvernance 20

13.1 Liste indicative d'endpoints API (REST) 20

13.2 Schéma de validation des entrées 20

13.3 Stratégie de déploiement des modèles 20

13.4 Glossaire 20

13.5 Responsabilités (vue simplifiée) 21

13.6 Perspectives 21

Chapitre 1

Signification du nom

1.1 Identité du produit

MOKENGELI désigne le nom commercial et technique de la plateforme d'intelligence artificielle prédictive anti-fraude développée dans le cadre du hackathon RawBank (finale). Elle prolonge la vision initiale d'un moteur de type « *shield* » (bouclier) : une barrière continue entre les clients légitimes et les comportements frauduleux ou à risque, avec une décision prise **au moment de l'opération** et non après coup par un traitement différé.

1.2 Ancrage RawBank et contexte

Le projet s'inscrit explicitement dans l'écosystème **RawBank** et le contexte bancaire en République démocratique du Congo. Les canaux visés incluent notamment le **mobile banking**, les virements, les paiements par carte, les retraits et les opérations initiées via application, web ou canaux alternatifs (USSD, agence). Cette ancrage impose de prendre en compte des spécificités locales : réseaux parfois instables, usage mixte des canaux, montants et habitudes horaires propres au marché, et nécessité de garder une **latence faible** pour ne pas dégrader l'expérience client.

1.3 Les trois notions clés

Protection continue
: Le système ne se limite pas à un contrôle ponctuel : il entretient une vision actualisée du risque à chaque transaction ou événement pertinent, en s'appuyant sur l'historique et sur les signaux de session.

Traitement *live*
: La décision est produite dans le flux transactionnel, avant la finalisation comptable de l'opération lorsque le processus métier le permet. Cela distingue MOKENGELI d'un simple reporting de fraude *a posteriori*.

Équilibre friction / sécurité
: L'objectif affiché est de bloquer ou challenger les opérations suspectes **avant** la sortie des fonds, tout en évitant de soumettre systématiquement les clients honnêtes à des étapes lourdes inutiles (voir fonctionnalité « step-up dynamique »).

En synthèse : MOKENGELI incarne une protection RawBank « en direct », avec scoring et décision au fil de l'eau sur chaque transaction traitée par le moteur.

Chapitre 2

Vision du projet et principe général

2.1 Mission fonctionnelle

MOKENGELI est un **moteur anti-fraude en temps réel**. À chaque opération prise en charge par le connecteur bancaire (virement, paiement carte, retrait, mobile banking, etc.), le système exécute une chaîne de traitements cohérente :

1. **Calculer** un ou plusieurs scores de risque (transaction, session, comportement) ;
2. **Décider** d'une action : autoriser, bloquer, demander une preuve supplémentaire (OTP, biométrie, etc.) ou placer le dossier en file d'analyste ;
3. **Alerter** le client et/ou la banque lorsque la politique métier ou la conformité l'exigent ;
4. **Apprendre** en réinjectant les retours (faux positifs, fraudes confirmées, validations manuelles) dans les jeux de données et les cycles de réentraînement.

2.2 Objectifs mesurables (indicateurs de succès)

Pour la phase hackathon puis pour une mise en production progressive, les indicateurs suivants permettent de suivre la valeur du moteur :

— **Latence p95/p99** du chemin « réception requête — décision » (objectif : ordre de quelques dizaines de millisecondes pour la partie scoring, hors latence réseau extrême).

— **Taux de blocage et de challenge** par canal et par segment client (à calibrer pour limiter les faux positifs).

— **Volume de fraude estimée évitée** (sur la base des montants bloqués et des cas confirmés par la suite).

— **Taux de faux positifs** remontés par les analystes ou les clients (à minimiser).

— **Couverture des motifs de décision** : répartition des `decision_reason_codes` pour expliquer les blocages et ajuster les règles.

2.3 Contraintes et principes directeurs

— **Séparation règles réglementaires / ML :** les interdictions légales ou politiques strictes ne doivent pas être « apprises » par erreur : elles restent dans une couche de règles explicites.

— **Explicabilité :** en cas de contestation, le système doit pouvoir fournir une piste d'explication (scores, motifs, principales features contributives via SHAP ou équivalent pour les modèles tabulaires).

— **Traçabilité :** chaque décision est historisée avec version de modèle et snapshot de features pour audit (voir chapitre sur la base de données).

Chapitre 3

Les trois modèles : rôle et fonctionnement

Trois modèles de machine learning se complètent pour alimenter la politique de décision. Cette architecture dépasse le schéma initial « deux modèles » (transaction + session) en ajoutant explicitement une couche **comportement titulaire**, alignée sur le dataset unifié décrit en section 3.4.

3.1 Modèle 1 — Scoring transactionnel (supervisé)

3.1.1 Rôle

Évaluer le risque de **fraude intrinsèque à la transaction** à partir de ses caractéristiques et du contexte financier immédiat (montant, contrepartie, vélocité, habitudes de montants, etc.).

3.1.2 Fonctionnement

Le modèle consomme un vecteur de features décrivant l'opération courante et ses agrégats récents. Il produit une **probabilité de fraude** (ex. $0{,}92$) ou un score calibré. Plus la probabilité est élevée, plus l'opération est classée comme suspecte au sens du label d'entraînement (`cible_fraude`).

3.1.3 Type de ML

On privilégie des modèles **interprétables et performants en tabulaire** : **Gradient Boosting** (XGBoost, LightGBM) ou **Forêts aléatoires**. Ces familles sont robustes aux données mixtes, gèrent bien les non-linéarités et permettent d'extraire l'importance des variables (permutation importance, SHAP).

3.1.4 Données d'entraînement

Historiques de transactions étiquetées `fraude` / `non_fraude` (données réelles ou **synthétiques** pour le hackathon). Les campagnes de feedback (faux positifs, fraudes confirmées) enrichissent progressivement ce jeu et permettent des réentraînements planifiés.

3.1.5 Variables importantes (exemples)

Montant par rapport aux habitudes, **vélocité** (nombre d'opérations sur fenêtre glissante), nouveauté du bénéficiaire, écart géographique ou temporel, cohérence canal / heure. L'importance relative est mesurée par **SHAP** ou **permutation importance** pour la communication aux métiers.

3.1.6 Sortie

Un **score transactionnel** (ex. échelle 0–100 ou probabilité calibrée), consommé par la couche de règles avec des seuils paramétrables (`policy_config`).

3.2 Modèle 2 — Détection d'anomalie de session

3.2.1 Rôle

Détecter si la **session** (connexion, navigation) est atypique par rapport au comportement habituel du titulaire, **même lorsque les identifiants sont corrects** — cas typique de prise de contrôle de session ou d'usage frauduleux d'un terminal.

3.2.2 Fonctionnement

Le modèle n'analyse pas uniquement une transaction isolée : il s'appuie sur des signaux de session : durée depuis la connexion, nombre et ordre des écrans visités, délai avant validation d'un OTP, type d'appareil, cohérence IP / pays, heure de connexion. Ces signaux sont comparés à un profil de normalité (par client ou par segment).

3.2.3 Type de ML

Approches **non supervisées ou semi-supervisées** : **Isolation Forest**, **One-Class SVM**, ou **autoencodeur** (reconstruction error élevée = anomalie) pour démontrer une piste « deep learning » si le temps le permet.

3.2.4 Données

Sessions labellisées *normale* / *compromise* lorsque disponibles ; sinon apprentissage sur **sessions normales uniquement** pour modéliser le comportement habituel et traiter les écarts comme anomalies.

3.2.5 Variables importantes

Délai OTP, séquence d'écrans, heure de connexion vs habitudes, changement d'appareil ou d'IP, nombre de tentatives de connexion échouées.

3.2.6 Sortie

Score d'anomalie de session (ex. 0–1). Une valeur élevée augmente le risque global même si le score transactionnel seul serait modéré.

3.3 Modèle 3 — Comportement du titulaire (profil habituel)

3.3.1 Rôle

Mesurer l'écart entre le **comportement actuel** (fréquence, montants, créneaux, lieux, types d'opérations) et le **profil historique** du client.

3.3.2 Fonctionnement

À partir d'agrégats par titulaire (rolling windows, médianes, distributions), le modèle évalue si l'usage courant « ressemble » à l'historique. Une forte déviation augmente le score, y compris lorsque la transaction isolée ne présente pas de montant extrême.

3.3.3 Type de ML

Supervisé si des labels « comportement frauduleux » existent ; sinon combinaison de **scores de distance** au profil (similarité, z-scores sur features dérivées) et de classifieurs légers.

3.3.4 Sortie

Score de risque comportemental (0–1 ou 0–100), combiné aux autres scores selon la politique.

3.4 Dataset unifié : 20 features et 3 cibles

3.4.1 Principe

Le jeu de données tabulaire aligne les trois modèles : chaque ligne correspond typiquement à une **transaction** enrichie du contexte de session au moment de l'événement. On compte **20 features** d'entrée et **3 cibles** (une par famille de modèle). Les cibles ne doivent **pas** être utilisées comme entrées en inférence réelle (risque de fuite de label).

3.4.2 Les trois cibles

Cible	Description
`cible_fraude`	Fraude sur l'opération (0/1) — supervision du Modèle 1.
`cible_session_anormale`	Session anormale ou compromise (0/1) — Modèle 2.
`cible_comportement_atypique`	Comportement du titulaire atypique (0/1 ou score terrain) — Modèle 3.

Remarque : un champ métier « statut normal / frauduleux » correspond en général à `cible_fraude` ; en entraînement c'est une **cible**, pas une feature.

3.4.3 Liste des 20 features

#	Nom	Description et intérêt pour le risque
1	`date_transaction`	Jour civil de l'opération ; capture saisonnalité, fins de mois, jours fériés.
2	`numero_transaction`	Identifiant unique pour jointures et traçabilité.
3	`id_client`	Clé titulaire ; base des agrégats historiques.
4	`montant`	Signal principal ; comparé à médiane / max habituels.
5	`localisation_zone`	Zone géographique ou agglomération ; détection d'écart au lieu habituel.
6	`heure`	0–23 ; opérations nocturnes atypiques pour le profil.
7	`type_transaction`	Virement, P2P, carte, retrait ; sensibilité au risque différente.
8	`type_appareil`	Mobile, web, tablette ; cohérence avec l'historique.
9	`canal`	App, web, USSD, agence ; friction et risque variables.
10	`jour_semaine`	Lundi–dimanche ; habitudes professionnelles vs week-end.
11	`vitesse_24h`	Nombre d'opérations sur 24 h ; détection de rafales.
12	`ratio_montant_median_30j`	Montant / médiane 30 jours ; pic brutal vs habitudes.
13	`beneficiaire_nouveau`	Indique un premier paiement récent vers la contrepartie.
14	`distance_km_habitude`	Écart au lieu d'usage habituel (km ou tranche).
15	`duree_session_min`	Temps entre connexion et validation ; sessions trop courtes ou longues.
16	`nb_ecrans_session`	Profondeur de navigation ; parcours atypique.
17	`delai_otp_s`	Délai entre envoi OTP et saisie ; détection d'automatisation ou d'hésitation.
18	`nb_echecs_login_24h`	Indicateur de credential stuffing ou brute force récent.
19	`changement_appareil`	Binaire : appareil différent du plus fréquent.
20	`ip_pays_inhabituel`	Cohérence pays IP avec profil (VPN, voyage, fraude).

3.4.4 Répartition indicative par modèle

Modèle 1 : priorité aux features financières et opérationnelles (montant, type, canal, vélocité, bénéficiaire, distance, IP). **Modèle 2** : signaux session / device / OTP / échecs login. **Modèle 3** : habitudes (heure, jour, vélocité, ratios, changements). Les features peuvent être **partagées** ; la cible change selon la tâche d'apprentissage.

3.4.5 Exemple de cinq enregistrements (synthétique)

numero	client	montant	type	M1	M2	M3	cfr	cs	cc
TXN-001	C-501	75 000	P2P	0.12	0.05	0.08	0	0	0
TXN-002	C-502	4 500 000	Vir.	0.87	0.78	0.72	1	1	1
TXN-003	C-501	82 000	Carte	0.14	0.06	0.09	0	0	0
TXN-004	C-503	200 000	Retrait	0.22	0.65	0.55	0	1	1
TXN-005	C-504	15 000	Facture	0.10	0.04	0.07	0	0	0

Lecture : les colonnes `c_fr`, `c_sess`, `c_comp` illustrent les trois cibles binaires possibles sur chaque ligne (exemple pédagogique). Les scores M1/M2/M3 sont des sorties modèle à comparer aux seuils métier.

Chapitre 4

Combinaison des trois modèles

4.1 Logique de fusion

Les sorties **score transactionnel**, **score session** et **score comportemental** sont agrégées selon une fonction de fusion paramétrable : moyenne pondérée, maximum pondéré, ou règles du type « si l'un des scores dépasse un plafond, forcer challenge ». La pondération peut dépendre du canal (ex. plus de poids au score session sur le canal web).

4.2 Exemples de politique (à calibrer)

— Score transaction très élevé **et** session suspecte ⇒ **blocage** ou file analyste prioritaire.

— Score transaction modéré, session normale, comportement cohérent ⇒ **OTP léger** ou autorisation avec alerte.

— Scores bas sur les trois axes ⇒ **autorisation** sans friction supplémentaire.

4.3 Rôle de la couche de règles métier

La **couche de règles** traduit le score combiné et les contraintes réglementaires en actions : `allow`, `block`, `challenge`. Elle produit aussi des `decision_reason_codes` pour l'audit et l'affichage analyste. Elle est configurable sans redéploiement de code lorsque les seuils sont externalisés en base (`policy_config`).

Chapitre 5

Structure de la base de données

5.1 Principe d'accès

Les applications React ne dialoguent **pas** directement avec PostgreSQL : seul le **backend Node** utilise la clé *service* Supabase. Cela réduit la surface d'attaque et centralise la logique métier.

5.2 Tables principales et rôle métier

`clients`
: Référentiel des titulaires avec identifiant externe (`C-501`, `DEMO_CLIENT_01`). Permet de rattacher sessions et transactions.

`sessions`
: Contexte de connexion pour le Modèle 2 : horodatage, device, canal, métadonnées JSON (parcours, IP si autorisé).

`transactions`
: Fait central : montant, type, bénéficiaire, liens `client_id`, `session_id`, champ `source` (`demo`, `sandbox`, `prod`), `raw_payload` pour rejeu.

`risk_scores`
: Une ligne par transaction évaluée : scores M1, M2, M3, score combiné, décision, codes motif, `features_snapshot`, `model_versions`, horodatage `computed_at`.

`analyst_feedback`
: Boucle d'apprentissage : type de feedback, commentaire, auteur, lien vers `transaction_id`.

`policy_config`
: Paramètres de seuils versionnés pour ajuster la sensibilité sans rebuild applicatif.

5.3 Relations cardinales

`clients` $1 : N$ `sessions` ; `clients` $1 : N$ `transactions` ; `sessions` $1 : N$ `transactions` ; `transactions` $1 : 1$ `risk_scores` ; `transactions` $1 : N$ `analyst_feedback`.

5.4 Colonnes clés rappelées

`transactions` : `numero_transaction`, `client_id`, `session_id`, `date_transaction`, `heure`, `montant`, `localisation_zone`, `type_transaction`, `canal`, `beneficiaire_ref`, `source`, `raw_payload`,

created_at.

risk_scores : transaction_id, score_m1_transaction, score_m2_session, score_m3_behavior, score_combined, decision, decision_reason_codes, features_snapshot, model_versions, computed_at.

Chapitre 6

Architecture de l'application

6.1 Backend (Node.js)

L'API HTTP expose notamment : évaluation de transaction, création/consultation de session, endpoints back-office (liste paginée, détail, saisie de feedback). Au démarrage, les **trois modèles** sont chargés en mémoire pour éviter un coût d'I/O par requête. Un module de **feature engineering** interroge Supabase pour vélocité, médianes, etc.

6.2 Frontend (React)

Deux SPA distinctes : **web-simulation** (client) et **web-admin** (analyste). Communication via JWT ou clé API selon le mode choisi ; aucun secret serveur dans le bundle.

6.3 Base Supabase

Un projet = une instance PostgreSQL ; extensions possibles (pgcrypto, etc.) selon besoins de conformité.

6.4 Sécurité dans la stack

Rappel : HTTPS, moindre privilège, pas d'accès direct DB depuis le navigateur. Détail au chapitre 12.

6.5 Organisation du dépôt (monorepo)

`api/`, `web-simulation/`, `web-admin/`, `packages/shared` (types TypeScript, schémas de validation Zod).

Chapitre 7

Fonctionnalités complètes (détail)

7.1 Fonctionnalité 1 — Scoring transactionnel en millisecondes

Comme dans les solutions de référence (FICO Falcon, Feedzai), MOKENGELI vise une décision en quelques **dizaines de millisecondes** pour le cœur du scoring. Pour y parvenir : pré-calcul d'agrégats (vélocité, médianes), mise en cache des lectures fréquentes, et modèles adaptés au temps réel. Le contexte congolais (montants, canaux, horaires, agents) est pris en compte dans le feature engineering.

7.2 Fonctionnalité 2 — Politique et file d'attente

Au-delà du score, la politique définit les seuils de blocage, de challenge et de mise en file analyste. Les **règles réglementaires** restent séparées des seuils ML pour permettre à la banque d'ajuster la sensibilité via configuration.

7.3 Fonctionnalité 3 — Alertes synchronisées

Comme en *alert banking*, le client reçoit une alerte alignée sur la décision : si blocage ou attente, SMS ou notification push avec message clair et possibilité de confirmer ou contester — réduction des litiges.

7.4 Fonctionnalité 4 — Boucle d'apprentissage

Les analystes et, le cas échéant, les clients étiquettent les cas (faux positif, fraude confirmée). Ces labels réinjectent le jeu d'entraînement pour réentraînements planifiés (auto-apprentissage maîtrisé).

7.5 Fonctionnalité 5 — Tableau de bord temps réel

Indicateurs : latence moyenne, nombre de blocages, taux de challenge, estimation de fraude évitée, répartition des motifs. Sert au **monitoring** et au pitch investisseur ou jury.

7.6 Fonctionnalité 6 — Score de session à la connexion (RAW Sentinel ID)

Avant la première transaction, évaluation de la session (device, heure, comportement). Si session suspecte, les opérations sensibles peuvent être soumises à validation renforcée — adresse le risque de **prise de contrôle de compte**.

7.7 Fonctionnalité 7 — Risque relationnel bénéficiaire (RAW Graph Vigil)

Enrichissement par graphe : le bénéficiaire est-il lié à des comptes ou réseaux déjà signalés ? Détection facilitée des **mules** et comptes relais même si les autres signaux sont modérés.

7.8 Fonctionnalité 8 — Step-up dynamique (RAW Trust Engine)

Réaction graduée selon le score combiné : faible → autorisation ; moyen → OTP/SMS ; élevé → blocage ou analyse. Paramétrable pour équilibrer sécurité et UX.

Chapitre 8

Interfaces utilisateur — vue d'ensemble

Deux applications distinctes consomment la même API : **simulation client** et **back-office**. Le détail des écrans figure au chapitre suivant.

Chapitre 9

Présentation des interfaces (fenêtres des applications)

9.1 Application client — simulation bancaire

Écrans A1 à A8 : accueil et choix de profil démo, connexion simulée, tableau de bord, formulaire de transaction, résultat MOKENGELI (autorisé / bloqué / challenge), saisie OTP, historique avec statuts colorés, zone d'alertes.

9.2 Application serveur — back-office

Écrans B1 à B9 : authentification analyste, dashboard KPI, fil des transactions avec scores M1/M2/M3, fiche détail avec `features_snapshot`, file d'attente priorisée, formulaire de feedback, vue client agrégée, paramètres `policy_config`, journal d'audit.

9.3 Cohérence

Les mêmes `numero_transaction` et décisions apparaissent côté client (historique) et analyste (fil) ; le détail technique reste principalement côté back-office.

Chapitre 10

Utilisation du logiciel et scénarios de test

10.1 Acteurs

Client (app bancaire), RawBank (back-office MOKENGELI), système (moteur transparent pour le client final).

10.2 Chaîne d'intégration

Le client initie une opération ; le back-end RawBank appelle MOKENGELI avant exécution ; la décision est appliquée et l'interface mise à jour.

10.3 Option A — Simulation complète (prioritaire)

Application factice, comptes démo, données synthétiques, appels API réels, affichage des décisions, feedback analyste. Aucun compte bancaire réel requis.

10.4 Option B — Sandbox RawBank

Branchement sur environnement de test si fourni par l'organisateur.

10.5 Option C — Démo hors ligne

Tableaux de bord préremplis, exports, présentation du code des modèles — filet de secours si intégration API incomplète.

Chapitre 11

Flux des données et exemple de traitement

11.1 Séquence runtime

1. Démarrage Node : chargement des modèles en RAM.
2. Réception de la requête d'évaluation.
3. Lecture BDD pour historique et agrégats.
4. Inférence M1, M2, M3.
5. Application des règles et écriture `transactions` + `risk_scores`.
6. Réponse JSON au client ou au connecteur RawBank.

Les modèles ne sont pas rechargés à chaque transaction.

11.2 Exemple chiffré (C-502)

Requête : virement **4 500 000** FC, bénéficiaire nouveau, heure atypique, session web courte. Scores indicatifs : M1 $= 0{,}87$, M2 $= 0{,}78$, M3 $= 0{,}72$, combiné $= 0{,}84 \Rightarrow$ décision `block` avec motifs stockés. Message client : opération bloquée, contacter la banque.

Chapitre 12

Sécurité

12.1 Transport et secrets

HTTPS obligatoire ; clés Supabase *service* et chemins modèles uniquement côté serveur ; variables d'environnement ou coffre de secrets.

12.2 Authentification et autorisation

Appels API authentifiés (JWT, clé API). Moindre privilège : droits différents pour simulation vs back-office.

12.3 Données et validation

Validation stricte des schémas JSON ; journalisation d'audit sans stocker OTP ou mots de passe en clair dans les logs applicatifs.

12.4 Résilience

Rate limiting sur endpoints sensibles ; messages d'erreur génériques côté client, détails côté logs serveur.

12.5 Chaîne ML

Intégrité des fichiers modèle (hash, versioning) ; traçabilité via `model_versions`.

12.6 PKI, mTLS et intégrité des payloads

Pour une cible production bancaire : infrastructure à clés publiques, authentification mutuelle TLS, signature des payloads pour limiter MITM et usurpation d'API. En hackathon, HTTPS + authentification API robuste peut suffire ; la feuille de route prévoit le durcissement progressif.

Chapitre 13

Annexes techniques et gouvernance

13.1 Liste indicative d'endpoints API (REST)

Les chemins suivants sont donnés à titre de spécification ; le préfixe `/api/v1` est un exemple.

`POST /evaluate/transaction`
: Corps JSON : identifiants client/session, montant, type, bénéficiaire, etc. Réponse : scores M1/M2/M3, décision, codes motif, identifiant de transaction persistée.

`GET /transactions`
: Liste paginée avec filtres (date, score min, décision) — réservé au back-office authentifié.

`GET /transactions/{id}`
: Détail complet incluant `features_snapshot` et lien session.

`POST /feedback`
: Saisie analyste : type de feedback, commentaire, référence transaction.

`GET /health`
: Sonde de disponibilité pour l'orchestration (load balancer, Kubernetes).

13.2 Schéma de validation des entrées

Toute requête doit passer par un validateur (schéma JSON Schema ou Zod côté Node) : types stricts, plages numériques (montant > 0), formats d'identifiants (`UUID`, préfixes démo). Les rejets sont journalisés sans exposer la cause exacte au client final.

13.3 Stratégie de déploiement des modèles

- Artefacts versionnés (`m1_v1.2.0.pkl`, etc.) stockés sur stockage objet ou disque sécurisé.
- Hash SHA-256 enregistré au build ; comparaison au chargement.
- Bascule *blue/green* ou *canary* possible si plusieurs versions coexistent (champ `model_versions` en base).

13.4 Glossaire

OTP One-Time Password — code à usage unique souvent envoyé par SMS.

SHAP
: Méthode d'explication des prédictions pour modèles tabulaires.

Step-up
: Renforcement progressif de l'authentification selon le risque.

mTLS
: TLS mutuel : client et serveur présentent un certificat.

Mule
: Compte intermédiaire utilisé pour recycler des fonds frauduleux.

13.5 Responsabilités (vue simplifiée)

Rôle	Responsabilité
Équipe Data / ML	Entraînement, évaluation, monitoring de dérive des modèles.
Backend	API, feature engineering, intégration Supabase, chargement modèles.
Frontend	Deux applications React, UX client et analyste.
Sécurité / Ops	Secrets, TLS, durcissement, sauvegardes.
Métier RawBank	Validation des seuils, interprétation des motifs, processus litiges.

13.6 Perspectives

Extension possible : moteur de règles déclaratif (Drools-like), intégration graphe de relations (*Graph Vigil*) hors MVP, LSTM sur séquences pour la démo avancée, et connexion au *core* bancaire réel en phase industrielle.

Fin du document — MOKENGELI / Brainsoft / RawBank Hackathon 2026.